

Practical: 6

Implementation of chain matrix multiplication using dynamic programming

01/09/2026

Aim: To Implement the chain matrix multiplication using dynamic programming

Algorithm:

Algorithm:

Input: n matrices and dimension array $P[0, \dots, n]$

Output: minimum no. of scalar multiplications required to multiply all matrices.

steps:

1. start
2. Read the number of matrix n
3. Read the dimension array: $P[0], P[1], \dots, P[n]$
4. Create two tables:
 $C[n+1][n+1]$
 $K[n+1][n+1]$
5. Initialize the cost table for a single matrix
 $C[i][i] = 0$
6. $d = 1$ to $n-1$ Repeat in for loop.
calculate the ending matrix
 $j = i + d$
7. if $(min > 99999)$ then $min = 99999$
8. for $x = i$ where $x < j$ do
 $q = C[i][x] + C[x+1][j] + P[i-1] * P[x] * P[j]$
9. if $(min > q)$ then do
 $min = q$
 $K[i][j] = x$
10. increment the x values
11. Assign $C[i][j] = min$
12. stop.

Program:

```
#include <iostream>
using namespace std;

int main(){
    int n = 5;

    int p[] = {5, 10, 3, 12, 5, 50};

    int c[10][10] = {0};
    int k[10][10] = {0};

    for(int d = 1; d < n; d++){
        for(int i = 1; i <= n-d; i++){
            int j = i + d;
            int min = 99999;

            for(int x = i; x < j; x++){
                int q = c[i][x] + c[x+1][j]
                    + p[i-1] * p[x] * p[j];

                if(q < min){
                    min = q;
                    k[i][j] = x;
                }
            }
            c[i][j] = min;
        }
    }

    cout << "\nMinimum cost = " << c[1][n]<<endl;

    return 0;
}
```

Output:

```
C:\Users\Chandu\OneDrive\D x + v

Minimum cost = 1655

-----
Process exited after 0.01228 seconds with return value 0
Press any key to continue . . . |
```

Time Complexity:

Time Complexity:-

```
int n=5;
int p[] = {5, 10, 3, 12, 5, 50};
int c[][][] = {0};
int k[][][] = {0};

for (int d=1; d<n; d++) {
    for (int i=1; i<=n-d; i++) {
        int j = i+d;
        int min = 99999;
        for (int x=i; x<j; x++) {
            j-i = d => d
            int q = c[i][x] + c[x+1][j] + p[i]*p[x+1]*p[j];
            if (q < min) {
                min = q;
                k[i][j] = x;
            }
        }
    }
}
```

$\rightarrow n-1$ [T(n) = O(n)]
 $\rightarrow n-d$ [O(n-1)]
 $\rightarrow O(n^2)$

$T(n) = \sum_{d=1}^{n-1} (n-d) \cdot d$
 $T(n) = \sum_{d=1}^{n-1} (nd - d^2)$
 $T(n) = \sum_{d=1}^{n-1} nd - \sum_{d=1}^{n-1} d^2$
 $\sum_{d=1}^{n-1} d = \frac{n(n-1)}{2}$
 $\Rightarrow d^2 = \left(\frac{n(n-1)}{2}\right)^2 \Rightarrow$
 $\sum_{d=1}^{n-1} d^2 = \frac{(n-1)n(2n-1)}{6}$
 $T(n) = n \cdot \frac{n(n-1)}{2} - \frac{(n-1)n(2n-1)}{6} = \frac{n(n-1)}{6} \cdot \frac{3n - (2n-1)}{1}$
 $= \frac{n(n-1)(n+1)}{6} \Rightarrow T(n) = \frac{n(n-1)(n+1)}{6}$
 $\Rightarrow \frac{n^3-n}{6} = n^2$
 $T(n) = \Theta(n^3)$

Note:-
 Best case: $O(n^3)$
 Average case: $O(n^3)$
 Worst case: $O(n^3)$

Conclusion:

The Implementation of the of chain matrix multiplication using dynamic programming has successfully executed with the time complexity of $O(n^3)$ in the Three cases of best, average and worst. At the end of this program says that the Minimum number of scalar multiplications required to multiply all matrices.